COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	October 5, 2026	**DATE SUBMITTED:**	October 5, 2026
NAME:	Gresola, Aryanna Keith G	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 10.1 **Graphs**

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

A.1. Create a Graph

INPUT

Graph.h

```
A1_GRAPH > C Graph.h > ...
  1  #ifndef GRAPH_H
  2  #define GRAPH_H
  3  #include <iostream>
  4
  5  struct Edge{
  6      int start;
  7      int end;
  8      int weight;
  9  };
 10
 11  class Graph{
 12  private:
 13      int vertices;
 14      Edge edges[100];
 15      int edgeCount;
 16
 17  public:
 18      Graph(int v){
 19          vertices = v;
 20          edgeCount = 0;
 21      }
```

```
23      void addVertex(){ // 1. add vertex
24          vertices++;
25      }
26
27      void addEdge(int start, int end, int weight){ // 2. add edge
28          if(edgeCount < 100){
29              edges[edgeCount].start = start;
30              edges[edgeCount].end = end;
31              edges[edgeCount].weight = weight;
32              edgeCount++;
33          }
34      }
35
36      void displayVertices(){ // 3. display graph vertices
37          std::cout << "Vertices: ";
38          for (int i = 0; i < vertices; i++){
39              std::cout << i << " ";
40          }
41      }
42
43      void displayEdges(){ // 3.1 display graph edges
44          std::cout << "Edges:" << std::endl;
45          for(int i = 0; i < edgeCount; i++){
46              std::cout << edges[i].start << " - " << edges[i].end
47              << " (weight: " << edges[i].weight << ")" << std::endl;
48          }
49      }
50  };
51
52  #endif
```

main.cpp

A1_GRAPH > main.cpp > ...

```
1   #include <iostream>
2   #include "Graph.h"
3
4   int main(){
5       Graph graph(6);
6
7       graph.addEdge(0, 1, 2);
8       graph.addEdge(0, 2, 4);
9       graph.addEdge(1, 4, 3);
10      graph.addEdge(2, 3, 2);
```

```
11      graph.addEdge(3, 1, 4);
12      graph.addEdge(4, 3, 3);
13
14      std::cout << "GRAPH IMPLEMENTATION: CREATE A GRAPH" << std::endl;
15      std::cout << std::endl;
16
17      graph.displayVertices();
18      std::cout << std::endl;
19
20      graph.displayEdges();
21
22      return 0;
23  }
```

OUTPUT

```
GRAPH IMPLEMENTATION: CREATE A GRAPH

Vertices: 0 1 2 3 4 5
Edges:
0 - 1 (weight: 2)
0 - 2 (weight: 4)
1 - 4 (weight: 3)
2 - 3 (weight: 2)
3 - 1 (weight: 4)
4 - 3 (weight: 3)
```

A.2. Adjacency Matrix

INPUT

AdjacencyMatrix.h

A2_AdjacencyMatrix > C AdjacencyMatrix.h > AdjacencyMatrix > ~AdjacencyMatrix()

```
1   #ifndef ADJACENCY_MATRIX_H
2   #define ADJACENCY_MATRIX_H
3   #include <iostream>
4
5   class AdjacencyMatrix{
6   private:
7       int N;
8       int** matrix;
```

```
10  public:
11      AdjacencyMatrix(int vertices){
12          N = vertices;
13
14          matrix = new int*[N];
15          for (int i = 0; i < N; i++){
16              matrix[i] = new int[N];
17
18              for (int j = 0; j < N; j++){
19                  matrix[i][j] = 0;
20              }
21          }
22      }
23
24      ~AdjacencyMatrix(){
25
26          for (int i = 0; i < N; i++){
27              delete[] matrix[i];
28          }
29
30          delete[] matrix;
31      }
32
33      void addEdge(int start, int end, int weight){
34
35          if (start >= 0 && start < N &&
36              end >= 0 && end < N){
37              matrix[start][end] = weight;
38          }
39      }
40
41      void display(){
42          std::cout << "ADJACENCY MATRIX" << std::endl;
43          std::cout << "    ";
44
45          for (int i = 0; i < N; i++){
46              std::cout << i << " ";
47          }
48          std::cout << std::endl;
49
50          for (int i = 0; i < N; i++){
51              std::cout << i << " | ";
```

```
52
53                  for (int j = 0; j < N; j++){
54                      std::cout << matrix[i][j] << " ";
55                  }
56                  std::cout << std::endl;
57              }
58          }
59      };
60
61      #endif
```

main.cpp

A2_AdjacencyMatrix > main.cpp > ...

```
1   #include <iostream>
2   #include "AdjacencyMatrix.h"
3
4   int main() {
5
6       const int N = 6;
7
8       AdjacencyMatrix graph(N);
9
10      graph.addEdge(0, 1, 2);
11      graph.addEdge(0, 2, 4);
12      graph.addEdge(1, 4, 3);
13      graph.addEdge(2, 3, 2);
14      graph.addEdge(3, 1, 4);
15      graph.addEdge(4, 3, 3);
16
17      graph.display();
18
19      return 0;
20  }
```

OUTPUT

```
ADJACENCY MATRIX
   0 1 2 3 4 5
0 | 0 2 4 0 0 0
1 | 0 0 0 0 3 0
2 | 0 0 0 2 0 0
3 | 0 4 0 0 0 0
4 | 0 0 0 3 0 0
5 | 0 0 0 0 0 0
```

A.3. Adjacency List

INPUT

DiaGraph.h

A3_AdjacencyList > C DiaGraph.h > ...

```
1   #ifndef DIAGRAPH_H
2   #define DIAGRAPH_H
3
4   #include <iostream>
5
6   // stores adjacency list items
7   struct adjNode{
8       int val, cost;
9       adjNode* next;
10  };
11
12  // structure to store edges
13  struct graphEdge{
14      int start_ver, end_ver, weight;
15  };
16
17  class DiaGraph{
18      adjNode* getAdjListNode(int value, int weight, adjNode* head){
19          adjNode* newNode = new adjNode;
20
21          newNode->val = value;
22          newNode->cost = weight;
23          newNode->next = head;
24
25          return newNode;
26      }
27
28      int N;
29
30  public:
31      adjNode **head;
32
33      DiaGraph(graphEdge edges[], int n, int N){
34
35          head = new adjNode*[N]();
36          this->N = N;
37
38          for (int i = 0; i < N; ++i)
39              head[i] = nullptr;
40
41          for (unsigned i = 0; i < n; i++){
42
```

```
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
            head[start_ver] = newNode;
        }
    }

    ~DiaGraph(){
        for (int i = 0; i < N; i++)
            delete[] head[i];

        delete[] head;
    }
};

#endif
```

main.cpp

A3_AdjacencyList > main.cpp > ...

```
#include <iostream>
#include "DiaGraph.h"

// print all adjacent vertices of a given vertex
void display_AdjList(adjNode* ptr, int i){
    std::cout << "Vertex " << i << ": ";

    if (ptr == nullptr){
        std::cout << "No edges";
    }else{
        while (ptr != nullptr) {
            std::cout << "(" << i << ", " << ptr->val << ", " << ptr->cost << ")";

            if (ptr->next != nullptr){
                std::cout << " -> ";
            }
            ptr = ptr->next;
        }
    }
    std::cout << std::endl;
}

// graph implementation
int main()
{
    graphEdge edges[] ={
```

```
27
28          // (x, y, w) -> edge from x to y with weight w
29          {0,1,2},
30          {0,2,4},
31          {1,4,3},
32          {2,3,2},
33          {3,1,4},
34          {4,3,3}
35      };
36
37      int N = 6; // number of vertices in the graph
38      // calculate number of edges
39      int n = sizeof(edges) / sizeof(edges[0]);
40      DiaGraph diagraph(edges, n, N);
41
42      std::cout << "GRAPH ADJACENCY LIST" << std::endl;
43      std::cout << "----------------------------------" << std::endl;
44      std::cout << "Format: (start_vertex, end_vertex, weight)" << std::endl;
45      std::cout << std::endl;
46
47      for (int i = 0; i < N; i++){
48          display_AdjList(diagraph.head[i], i);
49      }
50
51      return 0;
52  }
```

OUTPUT

```
GRAPH ADJACENCY LIST
----------------------------------
Format: (start_vertex, end_vertex, weight)

Vertex 0: (0, 2, 4) -> (0, 1, 2)
Vertex 1: (1, 4, 3)
Vertex 2: (2, 3, 2)
Vertex 3: (3, 1, 4)
Vertex 4: (4, 3, 3)
Vertex 5: No edges
```

ANALYSIS/OBSERVATION
In this activity, I observed how a graph is constructed and represented using C++. The graph was first created by defining its vertices and edges, then represented using an adjacency matrix and an adjacency list. I noticed that the adjacency matrix organizes the connections in rows and columns, while the adjacency list stores the connected vertices under each vertex. I also observed that the edge weights are included in both representations, which makes the relationship between the vertices clearer.

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

B.1. Depth-First Search

INPUT

DFS.h

B1_DFS > C DFS.h > ...

```cpp
#ifndef DFS_H
#define DFS_H
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const{
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const{
        return this->weight > e.weight;
    }
};
```

```cpp
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G){
    for (auto i = 1; i < G.vertices(); i++){
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges){
            os << "{" << e.dest << ": " << e.weight << "}, ";
        }
        os << std::endl;
    }
    return os;
}

// graph data structure
template <typename T>
class Graph{
public:
    Graph(size_t N) : V(N){}

    auto vertices() const{
        return V;
    }

    auto &edges() const{
        return edge_list;
    }

    void add_edge(Edge<T> &&e){
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V){
            edge_list.emplace_back(e);
        }else{
            std::cerr << "Vertex out of bounds" << std::endl;
        }
    }

    auto outgoing_edges(size_t v) const{
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list){
            if (e.src == v){
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

```

```
76      template <typename U>
77      friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
78
79  private:
80      size_t V;
81      std::vector<Edge<T>> edge_list;
82  };
83
84  template <typename T>
85  auto depth_first_search(const Graph<T> &G, size_t dest){
86      std::stack<size_t> stack;
87      std::vector<size_t> visit_order;
88      std::set<size_t> visited;
89
90      stack.push(1);
91      while (!stack.empty()){
92          auto current_vertex = stack.top();
93          stack.pop();
94
95          if (visited.find(current_vertex) == visited.end()){
96              visited.insert(current_vertex);
97              visit_order.push_back(current_vertex);
98
99              for (auto e : G.outgoing_edges(current_vertex)){
100                 if (visited.find(e.dest) == visited.end()){
101                     stack.push(e.dest);
102                 }
103             }
104         }
105     }
106     return visit_order;
107 }
```

```
108
109 template <typename T>
110 auto create_reference_graph(){
111     Graph<T> G(9);
112
113     std::map<unsigned, std::vector<std::pair<size_t, T >>> edges;
114
115     edges[1] = {{2, 0}, {5, 0}};
116     edges[2] = {{1, 0}, {5, 0}, {4, 0}};
117     edges[3] = {{4, 0}, {7, 0}};
118     edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
119     edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
120     edges[6] = {{4, 0}, {7, 0}, {8, 0}};
121     edges[7] = {{3, 0}, {6, 0}};
122     edges[8] = {{4, 0}, {5, 0}, {6, 0}};
123
```

```
124     for (auto &i : edges){
125         for (auto &j : i.second){
126             G.add_edge(Edge<T>{i.first, j.first, j.second});
127         }
128     }
129 
130     return G;
131 }
132 
133 #endif
```

main.cpp

```
B1_DFS > main.cpp > ...
  1  #include <iostream>
  2  #include "DFS.h"
  3  
  4  template <typename T>
  5  void test_DFS(){
  6      auto G = create_reference_graph<unsigned>();
  7  
  8      std::cout << "GRAPH:" << std::endl;
  9      std::cout << G << std::endl;
 10  
 11      std::cout << "DFS Order of vertices:" << std::endl;
 12  
 13      auto dfs_visit_order = depth_first_search(G, 1);
 14  
 15      for (auto v : dfs_visit_order){
 16          std::cout << v << std::endl;
 17      }
 18  }
 19  
 20  int main(){
 21      using T = unsigned;
 22      test_DFS<T>();
 23  
 24      return 0;
 25  }
```

OUTPUT

```
GRAPH:
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},
```

```
DFS Order of vertices:
1
5
8
6
7
3
4
2
```

B.2. Breadth-First Search

INPUT

BFS.h

B2_BFS > C BFS.h > ...

```
#ifndef BFS_H
#define BFS_H
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const{
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const{
        return this->weight > e.weight;
    }
};
```

```cpp
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G){
    for (auto i = 1; i < G.vertices(); i++){
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges){
            os << "{" << e.dest << ": " << e.weight << "}, ";
        }
        os << std::endl;
    }
    return os;
}

template <typename T>
class Graph{
public:
    Graph(size_t N) : V(N){}

    auto vertices() const{
        return V;
    }

    auto &edges() const{
        return edge_list;
    }

    void add_edge(Edge<T> &&e){
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V){
            edge_list.emplace_back(e);
        }else{
            std::cerr << "Vertex out of bounds" << std::endl;
        }
    }

    auto outgoing_edges(size_t v) const{
        std::vector<Edge<T>> edges_from_v;

        for (auto &e : edge_list){
            if (e.src == v){
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    friend std::ostream &operator<< <>(std::ostream &os, const Graph<T> &G);
```

```
76  private:
77      size_t V;
78      std::vector<Edge<T>> edge_list;
79  };
80
81  template <typename T>
82  auto create_reference_graph(){
83      Graph<T> G(9);
84
85      std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
86
87      edges[1] = {{2, 2}, {5, 3}};
88      edges[2] = {{1, 2}, {5, 5}, {4, 1}};
89      edges[3] = {{4, 2}, {7, 3}};
90      edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
91      edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
92      edges[6] = {{4, 4}, {7, 4}, {8, 1}};
93      edges[7] = {{3, 3}, {6, 4}};
94      edges[8] = {{4, 5}, {5, 3}, {6, 1}};
95
96      for (auto &i : edges){
97          for (auto &j : i.second){
98              G.add_edge(Edge<T>{i.first, j.first, j.second});
99          }
100     }
101     return G;
102 }
103
104 template <typename T>
105 auto breadth_first_search(const Graph<T> &G, size_t dest){
106     std::queue<size_t> queue;
107     std::vector<size_t> visit_order;
108     std::set<size_t> visited;
109
110     queue.push(1);
111     while (!queue.empty()){
112         auto current_vertex = queue.front();
113         queue.pop();
114
115         if (visited.find(current_vertex) == visited.end()){
116             visited.insert(current_vertex);
117             visit_order.push_back(current_vertex);
118
119             for (auto e : G.outgoing_edges(current_vertex)){
120                 queue.push(e.dest);
121             }
```

```
122             }
123         }
124         return visit_order;
125     }
126
127     #endif
```

main.cpp

B2_BFS > main.cpp > ...

```cpp
#include <iostream>
#include "BFS.h"

template <typename T>
void test_BFS(){
    auto G = create_reference_graph<unsigned>();

    std::cout << "GRAPH:" << std::endl;
    std::cout << G << std::endl;

    std::cout << "BFS Order of vertices:" << std::endl;

    auto bfs_visit_order = breadth_first_search(G, 1);

    for (auto v : bfs_visit_order){
        std::cout << v << std::endl;
    }
}

int main(){
    using T = unsigned;

    test_BFS<T>();

    return 0;
}
```

OUTPUT

```
GRAPH:
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},
```

```
BFS Order of vertices:
1
2
5
4
8
3
6
7
```

ANALYSIS/OBSERVATION

In this activity, I gained a better understanding of how graph traversal works by implementing DFS and BFS in C++. I observed that although both algorithms are used to visit the vertices of a graph, they follow different ways of exploring the connections. DFS focuses on following one path as far as possible before backtracking, while BFS explores the nearby vertices first before moving to the next level. I also noticed that the data structure used affects the traversal process, since DFS uses a stack while BFS uses a queue. Using a visited set in both algorithms is also important because it keeps track of the vertices that have already been explored and prevents unnecessary repeated visits. Through the activity, I understood that choosing between DFS and BFS depends on how the graph needs to be explored and how the connections between its vertices should be processed.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

QUESTIONS AND ANSWERS

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

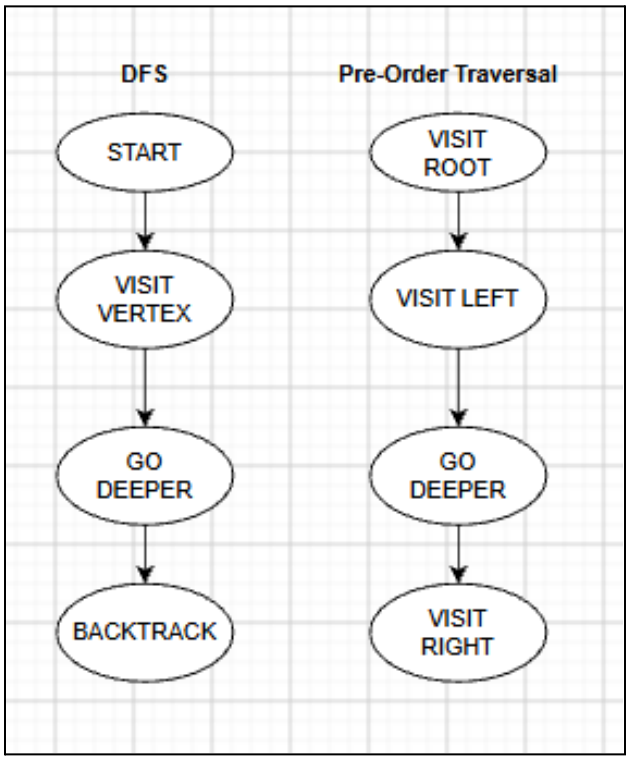
1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task

    - **The Depth-First Search (DFS) algorithm would be most helpful because it explores one path as deeply as possible before backtracking. For example, it is like entering one road and continuing forward until there are no more places to visit, then going back to the previous location and taking another unvisited road. This process continues until all reachable locations are visited.**

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

    - **The equivalent of DFS in tree traversal is Pre-order Traversal. Both explore a node first before going deeper into its branches. For example, in a tree, pre-order visits the root first, then explores the left side before moving to the right side, following the pattern Root - Left - Right.**

Comparison:



Code Implementation:

DFS	PRE-ORDER TRAVERSAL

`for (auto e : G.outgoing_edges(current_vertex)){`
`  if (visited.find(e.dest) == visited.end()){`
`  stack.push(e.dest);`
	`void preorder(Node* node){`
	`  if (node == nullptr)`
	`  return;`
	`  std::cout << node->data << " ";`
	`  preorder(node->left);`
	`  preorder(node->right);`
	`}`

Pseudocode:

DFS	PRE-ORDER TRAVERSAL
DFS(vertex) Mark vertex as visited For each unvisited vertex adjacent to vertex DFS(adjacent vertex)	Preorder(node) If node is not NULL Visit node Preorder(left child) Preorder(right child)

3. In the performed code, what data structure is used to implement the Breadth First Search?

   - **The queue data structure is used to implement BFS. It works like a line of people waiting where the first vertex added to the queue is the first one processed. This allows BFS to explore the graph level by level.**

4. How many times can a node be visited in the BFS?
   - **A node can be visited only once in BFS. The visited set keeps track of the vertices that have already been processed, so the same vertex will not be processed again even if it is connected to multiple vertices.**

CONCLUSION/LEARNING

In this activity, I learned how graphs can be implemented and represented in C++ using an adjacency matrix and adjacency list in ILO A. In ILO B, I also learned how DFS and BFS are used to traverse a graph and how they differ in the way they explore vertices. Although I still struggle to fully grasp the overall concept of graphs and their different operations, I was able to understand the basic ideas and how they are implemented in code. I know that I still have a lot to learn and more concepts to understand, but this activity helped me build a better foundation and gave me a clearer idea of how graph algorithms work.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar correction, code assistance, and explanation of concepts.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I used ChatGPT for grammar correction, code assistance, and concept explanations. It did not replace my original thinking, analysis, authorship, or completion of the required work.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.